

Phase 1: JavaScript Mastery - Module 3: Control Flow

Let's start module 3: Control Flow and the first topic is Conditional Statements - 'if-else', 'switch', 'ternary operator'

Module 3: Control Flow

Topic 1: Conditional Statements (if-else, switch, Ternary Operator)

Control Flow means **controlling the order in which a program executes statements**.

Imagine you're building an e-commerce website:

- If the user is logged in → Show dashboard.
- Else → Show login page.
- If the payment is successful → Show order confirmation.
- Else → Show payment failed message.

JavaScript uses **conditional statements** to make decisions.

1. if Statement

Used when you want to execute code only if a condition is true.

Syntax

```
JavaScript
if (condition) {
  // code executes if condition is true
}
```

Example

```
JavaScript
let age = 20;

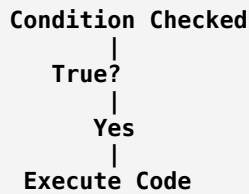
if (age >= 18) {
  console.log("You can vote");
}
```

Output

```
JavaScript
```

You can vote

Flow



2. if-else Statement

When you want one block to run if the condition is true and another if it is false.

Syntax

```

JavaScript
if (condition) {
  // true block
} else {
  // false block
}
  
```

Example

```

JavaScript
let age = 16;

if (age >= 18) {
  console.log("Eligible to vote");
} else {
  console.log("Not eligible to vote");
}
  
```

Output

```

JavaScript
Not eligible to vote
  
```

3. if-else if-else Ladder

Used when multiple conditions need to be checked.

Example

```

JavaScript
let marks = 75;

if (marks >= 90) {
  console.log("Grade A");
}
else if (marks >= 80) {
  console.log("Grade B");
}
  
```

```
else if (marks >= 70) {
    console.log("Grade C");
}
else {
    console.log("Fail");
}
```

Output

```
JavaScript
Grade C
```

Flow

```
Condition 1
|
True? ---> Execute Block 1
|
False
|
Condition 2
|
True? ---> Execute Block 2
|
False
|
Condition 3
|
True? ---> Execute Block 3
|
False
|
Else Block
```

Important Point

As soon as one condition becomes **true**, JavaScript stops checking the remaining conditions.

Truthy and Falsy Values

Conditions don't always have to be boolean values.

JavaScript automatically converts values into true or false.

Falsy Values

Only these values are falsy:

```
JavaScript
false
0
-0
0n
""
null
undefined
NaN
```

Example

```
JavaScript
let username = "";

if (username) {
  console.log("Welcome");
} else {
  console.log("Please enter username");
}
```

Output

```
JavaScript
Please enter username
```

Because an empty string "" is falsy.

Truthy Values

Everything else is truthy.

Examples:

```
JavaScript
"Hello"
[]
{}
42
-5
"0"
>false"
```

Example

```
JavaScript
if ("Hello") {
  console.log("Executed");
}
```

Output:

```
JavaScript
Executed
```

4. Switch Statement

Used when there are many possible values for a single variable.

Instead of writing many **if-else** statements, **switch** can be cleaner.

Syntax

```
JavaScript
switch (expression) {
  case value1:
    // code
    break;

  case value2:
    // code
    break;

  default:
    // code
}
```

Example

```
JavaScript
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;

  case 2:
    console.log("Tuesday");
    break;

  case 3:
    console.log("Wednesday");
    break;

  default:
    console.log("Invalid Day");
}
```

Output

```
JavaScript
Wednesday
```

Why break is Important?

Without **break**, execution continues into the next cases.

Example

```
JavaScript
let day = 1;

switch(day) {
  case 1:
    console.log("Monday");

  case 2:
    console.log("Tuesday");

  case 3:
    console.log("Wednesday");
}
```

Output

```
JavaScript
Monday
Tuesday
Wednesday
```

This behavior is called **Fall Through**.

Multiple Cases Together

```
JavaScript
let fruit = "apple";

switch (fruit) {
  case "apple":
  case "mango":
  case "banana":
    console.log("Fruit Found");
    break;

  default:
    console.log("Unknown Fruit");
}
```

Output:

```
JavaScript
Fruit Found
```

switch Uses Strict Equality (===)

JavaScript compares using strict equality.

Example

```
JavaScript
let num = "1";

switch (num) {
  case 1:
    console.log("Number");
    break;

  default:
    console.log("No Match");
}
```

Output:

```
JavaScript
No Match
```

Because:

```
JavaScript
"1" === 1
```

is false.

5. Ternary Operator (? :)

The ternary operator is a shorthand version of **if-else**.

Syntax

```
JavaScript
condition ? valueIfTrue : valueIfFalse;
```

Example

Using **if-else**

```
JavaScript
let age = 20;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Using ternary operator:

```
JavaScript
let age = 20;

console.log(age >= 18 ? "Adult" : "Minor");
```

Output:

```
JavaScript
Adult
```

Storing Result

```
JavaScript
let age = 15;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

```
JavaScript
```

Nested Ternary

```
JavaScript
let marks = 85;

let grade =
  marks >= 90 ? "A" :
  marks >= 80 ? "B" :
  marks >= 70 ? "C" :
  "Fail";

console.log(grade);
```

Output:

```
JavaScript
B
```

⚠️ Avoid deeply nested ternary operators because they reduce readability.

When to Use What?

Situation	Best Choice
Single condition	if
Two outcomes	if-else
Multiple conditions	if-else if
Many exact values of one variable	switch
Short simple condition	Ternary Operator

Interview Questions

Q1. Difference between == and === in conditions?

```
JavaScript
5 == "5"    // true
5 === "5"   // false
```

== performs type coercion.

=== checks both value and type.

Q2. Which values are falsy in JavaScript?

```
JavaScript
```

```
false
0
-0
0n
...
null
undefined
NaN
```

Q3. Why is break used in switch?

To stop execution after the matching case. Without it, fall-through occurs.

Q4. Is ternary operator faster than if-else?

Practically, no noticeable difference. Choose based on readability.

Quick Revision

```
JavaScript
// if
if (condition) { }

// if-else
if (condition) { }
else { }

// if-else if
if (condition1) { }
else if (condition2) { }
else { }

// switch
switch(value) {
  case x:
    break;
  default:
}

// ternary
condition ? trueValue : falseValue;
```

Real Rule to Remember

- Decision based on a condition → if-else
- Decision based on one variable having many fixed values → switch
- Short one-line decision → Ternary Operator

Topic 2: Loops - 'for', 'while', 'do-while'

Module 3: Control Flow

Topic 2: Loops (for, while, do-while)

Imagine you want to print:

```
Hello  
Hello  
Hello  
Hello  
Hello
```

Without loops:

```
JavaScript  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");
```

This is repetitive. What if you need to print it 1000 times?

That's where **loops** come in.

A **loop** allows a block of code to execute repeatedly until a condition becomes false.

Why Loops Are Important?

Real-world examples:

- Showing 100 products on an e-commerce website.
 - Processing thousands of user records.
 - Iterating through arrays.
 - Downloading multiple files.
 - Sending notifications to many users.
-

Types of Loops in JavaScript

1. **for** Loop
 2. **while** Loop
 3. **do-while** Loop
-

1. for Loop

Most commonly used loop.

Syntax

```
JavaScript
```

```
for(initialization; condition; update) {
  // code
}
```

Components

```
JavaScript
for(let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Breakdown

```
JavaScript
let i = 1;      // Initialization
i <= 5;         // Condition
i++;            // Update
```

Execution Flow

```
Initialization
  ↓
Condition
  ↓
True?
  ↓
Execute Block
  ↓
Update
  ↓
Back to Condition
```

Example: Print 1 to 5

```
JavaScript
for(let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Output:

```
1
2
3
4
5
```

Example: Print Even Numbers

```
JavaScript
for(let i = 2; i <= 10; i += 2) {
  console.log(i);
}
```

Output:

```
2
4
6
8
10
```

Example: Multiplication Table of 5

JavaScript

```
for(let i = 1; i <= 10; i++) {
  console.log(`5 x ${i} = ${5 * i}`);
}
```

Output:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Loop Variable (i)

Conventionally:

JavaScript

```
i = index
j = second loop
k = third loop
```

Example:

JavaScript

```
for(let i = 0; i < 3; i++) {
  console.log(i);
}
```

Output:

```
0
1
2
```

Infinite Loop

If the condition never becomes false:

JavaScript

```
for(let i = 1; i > 0; i++) {
  console.log(i);
}
```

This runs forever because:

```
JavaScript
i > 0
```

always remains true.

⚠ Infinite loops can freeze your browser or application.

2. while Loop

Used when the number of iterations is not known beforehand.

Syntax

```
JavaScript
while(condition) {
  // code
}
```

Example

```
JavaScript
let i = 1;

while(i <= 5) {
  console.log(i);
  i++;
}
```

Output:

```
1
2
3
4
5
```

Flow

```
Condition
  ↓
True?
  ↓
Execute Block
  ↓
Update
  ↓
Back to Condition
```

Real-World Example

Imagine a game:

```
JavaScript
let lives = 3;

while(lives > 0) {
  console.log("Player is alive");
  lives--;
}
```

Output:

```
Player is alive
Player is alive
Player is alive
```

Example: Sum of Numbers

```
JavaScript
let sum = 0;
let i = 1;

while(i <= 5) {
  sum += i;
  i++;
}

console.log(sum);
```

Output:

```
15
```

Calculation:

```
1 + 2 + 3 + 4 + 5 = 15
```

3. do-while Loop

Special loop.

The code executes **at least once**, even if the condition is false.

Syntax

```
JavaScript
do {
  // code
}
while(condition);
```

Example

```
JavaScript
let i = 1;

do {
  console.log(i);
  i++;
}
while(i <= 5);
```

Output:

```
1
2
3
4
5
```

Difference Between while and do-while

While Loop

```
JavaScript
let i = 10;

while(i < 5) {
  console.log(i);
}
```

Output:

```
Nothing
```

Because condition is false from the start.

Do-While Loop

```
JavaScript
let i = 10;

do {
  console.log(i);
}
while(i < 5);
```

Output:

```
10
```

The block executes once before checking the condition.

Real-World Example

Login attempt system:

```
JavaScript
let passwordCorrect = false;

do {
  console.log("Enter Password");
}
while(!passwordCorrect);
```

The user must get at least one chance to enter a password.

Comparing All Three Loops

Feature	for	while	do-while
Initialization inside loop	✓	✗	✗
Condition checked first	✓	✓	✗
Executes at least once	✗	✗	✓
Best when iteration count known	✓	✗	✗
Best when iteration count unknown	✗	✓	✓

When Should You Use Each?

Use for

When you know exactly how many times the loop should run.

```
JavaScript
for(let i = 1; i <= 10; i++)
```

Examples:

- Print 1–100
- Traverse an array
- Generate a multiplication table

Use while

When the number of iterations is unknown.

```
JavaScript
while(userLoggedIn)
```

Examples:

- Game loops
 - Waiting for user input
 - Reading data until a condition is met
-

Use do-while

When the code must run at least once.

```
JavaScript
do {
  // menu
}
while(choice !== "exit");
```

Examples:

- ATM menu
 - Login system
 - Retry mechanisms
-

Common Mistakes

1. Forgetting Update

```
JavaScript
let i = 1;
while(i <= 5) {
  console.log(i);
}
```

Output:

Infinite Loop

Because `i` never changes.

2. Wrong Condition

```
JavaScript
for(let i = 1; i >= 1; i++)
```

Infinite loop because condition is always true.

3. Semicolon Mistake

```
JavaScript
for(let i = 1; i <= 5; i++);

{
  console.log("Hello");
}
```

The loop body becomes empty due to the semicolon.

Interview Questions

Q1. Difference between while and do-while?

while checks condition first.

do-while executes once and then checks condition.

Q2. Which loop is used most often?

for loop.

Because initialization, condition, and update are in one place.

Q3. Can a for loop be converted to a while loop?

Yes.

```
JavaScript
for(let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Equivalent:

```
JavaScript
let i = 1;

while(i <= 5) {
  console.log(i);
  i++;
}
```

Quick Revision

```
JavaScript
// FOR LOOP
for(let i = 1; i <= 5; i++) {
  console.log(i);
}

// WHILE LOOP
```

```
let i = 1;
while(i <= 5) {
  console.log(i);
  i++;
}

// DO-WHILE LOOP
let j = 1;
do {
  console.log(j);
  j++;
}
while(j <= 5);
```

Golden Rule

- **Known number of iterations** → `for`
- **Unknown number of iterations** → `while`
- **Must execute at least once** → `do-while`

These three loops form the foundation for everything you'll do later in JavaScript, including arrays, objects, DOM manipulation, APIs, and React.

Topic 3: Break and Continue

Module 3: Control Flow

Topic 3: break and continue

While working with loops, there are situations where you want to:

- Stop the loop completely.
- Skip a particular iteration and move to the next one.

JavaScript provides two keywords for this:

1. `break`
2. `continue`

1. break Statement

The `break` statement **immediately terminates the loop**.

When JavaScript encounters `break`, it exits the loop and continues executing the code after the loop.

Syntax

```
JavaScript
for(...) {
  if(condition) {
    break;
  }
}
```

Example: Stop at 5

```
JavaScript
for(let i = 1; i <= 10; i++) {
  if(i === 5) {
    break;
  }

  console.log(i);
}
```

Output

```
1
2
3
4
```

Execution Flow

```
i = 1 → Print
i = 2 → Print
i = 3 → Print
i = 4 → Print
i = 5 → break
Loop Ends
```

Real-World Example

Imagine searching for a product in a list.

```
JavaScript
let products = ["Laptop", "Mouse", "Keyboard", "Monitor"];

for(let i = 0; i < products.length; i++) {
  if(products[i] === "Keyboard") {
    console.log("Product Found");
    break;
  }
}
```

Once found, there's no need to continue searching.

break in a while Loop

```
JavaScript
let i = 1;
```

```
while(true) {
  console.log(i);

  if(i === 5) {
    break;
  }

  i++;
}
```

Output

```
1
2
3
4
5
```

The loop would otherwise run forever because:

```
JavaScript
while(true)
```

is always true.

2. continue Statement

The `continue` statement **skips the current iteration** and moves directly to the next iteration.

The loop itself does not stop.

Syntax

```
JavaScript
for(...) {
  if(condition) {
    continue;
  }
}
```

Example: Skip 5

```
JavaScript
for(let i = 1; i <= 10; i++) {
  if(i === 5) {
    continue;
  }

  console.log(i);
}
```

Output

```
1
2
3
4
6
7
8
9
10
```

Notice that only 5 is skipped.

Execution Flow

```
i = 1 → Print
i = 2 → Print
i = 3 → Print
i = 4 → Print
i = 5 → continue
Skip Print
Move to next iteration
i = 6 → Print
```

Real-World Example

Suppose you want to process all users except banned users.

```
JavaScript
let users = ["Aman", "Ravi", "BannedUser", "Krishna"];

for(let i = 0; i < users.length; i++) {
    if(users[i] === "BannedUser") {
        continue;
    }
    console.log("Processing:", users[i]);
}
```

Output

```
Processing: Aman
Processing: Ravi
Processing: Krishna
```

Difference Between break and continue

Feature	break	continue
Stops loop completely	✓	✗
Skips current iteration	✗	✓
Continues remaining iterations	✗	✓
Exits loop immediately	✓	✗

Example Comparison

Using break

```
JavaScript
for(let i = 1; i <= 10; i++) {
    if(i === 5) {
        break;
    }
    console.log(i);
}
```

Output:

```
1
2
3
4
```

Using continue

```
JavaScript
for(let i = 1; i <= 10; i++) {
    if(i === 5) {
        continue;
    }
    console.log(i);
}
```

Output:

```
1
2
3
4
6
7
8
9
10
```

break in a switch

You've already seen this in the `switch` statement.

```
JavaScript
let day = 2;
switch(day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
}
```

```
        break;
    default:
        console.log("Invalid Day");
}
```

Without **break**, execution falls through to the next case.

Nested Loops and break

Consider:

```
JavaScript
for(let i = 1; i <= 3; i++) {
    for(let j = 1; j <= 3; j++) {
        if(j === 2) {
            break;
        }
        console.log(i, j);
    }
}
```

Output

```
1 1
2 1
3 1
```

Why?

When **j** becomes 2, only the **inner loop** stops.

The outer loop continues.

Labeled break

JavaScript allows labels to break out of outer loops.

```
JavaScript
outerLoop:
for(let i = 1; i <= 3; i++) {
    for(let j = 1; j <= 3; j++) {
        if(i === 2 && j === 2) {
            break outerLoop;
        }
        console.log(i, j);
    }
}
```

Output

```
1 1
1 2
1 3
2 1
```

The entire loop structure stops.

⚠ Labeled statements are rarely used in modern JavaScript.

Common Use Cases

Find First Match

```
JavaScript
for(let i = 0; i < arr.length; i++) {
  if(arr[i] === target) {
    break;
  }
}
```

Use `break`.

Skip Invalid Data

```
JavaScript
for(let i = 0; i < data.length; i++) {
  if(data[i] === null) {
    continue;
  }
  console.log(data[i]);
}
```

Use `continue`.

Common Mistake

Forgetting Update in `while` with `continue`

✗ Wrong

```
JavaScript
let i = 0;
while(i < 5) {
  if(i === 2) {
    continue;
  }
  console.log(i);
}
```

```
i++;
}
```

What Happens?

When `i` becomes 2:

```
JavaScript
continue;
```

runs before:

```
JavaScript
i++;
```

So `i` remains 2 forever.

Result:

Infinite Loop

Correct Version

```
JavaScript
let i = 0;
while(i < 5) {
    i++;
    if(i === 2) {
        continue;
    }
    console.log(i);
}
```

Interview Questions

Q1. What is the difference between `break` and `continue`?

- `break` exits the loop completely.
- `continue` skips the current iteration and proceeds to the next.

Q2. Can `break` be used in a `switch` statement?

Yes. It prevents fall-through.

Q3. Does `continue` stop the loop?

No. It only skips one iteration.

Q4. Can break be used in for, while, and do-while loops?

Yes.

Quick Revision

```
JavaScript
// break
for(let i = 1; i <= 10; i++) {
    if(i === 5) {
        break;
    }
    console.log(i);
}
```

Output:

```
1 2 3 4
```

```
JavaScript
// continue
for(let i = 1; i <= 10; i++) {
    if(i === 5) {
        continue;
    }
    console.log(i);
}
```

Output:

```
1 2 3 4 6 7 8 9 10
```

Memory Trick

Think of a race:

🚩 **break** = Leave the race completely.

⏭️ **continue** = Skip one checkpoint and keep running.

This simple idea will help you remember the difference forever.